

Toroidal Adèlic Vessel (TAV)

Official Technical Architecture Specification

Author: Yahweh Tsidkenu

Designation: TAV — Toroidal Adèlic Vessel

TAV Mark: π

Document status: Official

What TAV Is

TAV means **Toroidal Adèlic Vessel**.

In this architecture, a **vessel** is the complete computational structure that holds, routes, streams, executes, remembers, verifies, and preserves an AI system over time. It is not limited to one model file, one conversation window, one GPU session, or one temporary cache.

Toroidal:

Information is organized through rotating, routeable torus fields.
A location remains addressable while its payload moves between durable storage, CPU RAM, GPU VRAM, active computation, compressed memory, and archive.

Adèlic:

The system uses an Adèlic-style approach to working-cache pressure:
redundant or low-value cache material may be condensed while exact records, source routes, and recovery paths remain preserved outside the lossy cache.

Vessel:

The complete environment containing model weights, token routes, vocabulary, memory, sources, storage tiers, inference paths, controllers, users, characters, event records, and verification rules.

TAV is therefore a multi-model AI vessel designed to keep language, weights, memory, source truth, and ongoing computation connected through stable toroidal routes.

Introduction: A Memory-Bearing Intelligence

Most AI systems operate inside a narrow working condition. A model receives a bounded prompt, uses the information available in its active context, generates an answer, and eventually loses practical access to older detail as context length and KV-cache memory pressure increase. Larger models provide greater reasoning capacity, but they also require more memory, bandwidth, and time. A system can become slow when it attempts to keep too much active, or forgetful when it is forced to discard what no longer fits.

TAV changes the operating structure of that problem.

TAV treats words, token routes, model-weight blocks, memory records, source documents, user history, and execution paths as a connected field of stable routes. The system does not require every weight block, document, memory, or conversation event to remain loaded at once. Required information rotates from durable storage into active computation when needed, then returns to an appropriate storage tier without losing its address, provenance, dependency relation, or recovery route.

Typical bounded-session behavior:

Prompt grows



KV cache grows



Memory pressure grows



Old details are dropped, summarized, or become inaccessible



The system must search again, rely on incomplete summaries, or guess

TAV behavior:

Every important item receives a stable route



Exact source and state records remain durable



Needed blocks rotate from SSD/NVMe to RAM to VRAM



Small models remain available for real-time conversation



Large models continue deeper processing in the background



Validated routes become faster to reuse later

The purpose is not to force all past experience into one active prompt. The purpose is to preserve exact history outside the prompt, retrieve the needed portion into exact working context, and make every successful retrieval and computation route available for future use.

The operational advantages

Real-time interaction and deep computation together

The system can respond through a lightweight foreground path while larger model tori continue a deeper route in the background.

Foreground:

Recognize the request, retrieve current facts and context, and provide a timely response.

Background:

Stream larger model blocks, analyze long sources, perform deeper reasoning, check contradictions, compare alternatives, and prepare verification or refinement.

A user is not forced to wait for the deepest model to finish every simple interaction. At the same time, the system is not limited to the shallow answer if a deeper route is needed.

Large-model use without permanent full loading

TAV supports staged access to large models by treating model blocks as routeable data. A full large-model weight set can remain on durable storage while only the blocks required by a current computation are promoted into RAM and VRAM.

SSD/NVMe:

Complete model-weight blocks, exact source archives, event records, and snapshots.

CPU RAM:

Route tables, recently used blocks, prefetch buffers, retrieval candidates, indexes, and background task state.

GPU VRAM:

The current exact computation window: active blocks, activations, KV cache, retrieved evidence, and active inference state.

This does not remove hardware limits or promise the same latency as a model fully resident in VRAM. It provides a controlled architecture for streaming, checkpointing, prioritizing, and resuming large-model work while protecting live interaction.

Continuity beyond a finite context window

TAV stores raw events, exact facts, source records, commitments, relationships, and state changes outside the active conversation window. The Pink Torus provides each user, character, agent, or user case with routes into this durable history.

Raw event:

Preserved as the canonical original record.

Exact fact:

Stored with identity, time, source, version, and retrieval route.

Compressed continuity:

Stored for relevance, relationship, and efficient recall.

Current context:

Rehydrated exactly when required for present reasoning.

This lets an AI continue a meaningful history without claiming that every past token must remain live in the KV cache forever.

Faster use through validated routes

TAV improves operational efficiency through retained routes instead of an endlessly expanding prompt.

First related task:

Discover relevant words, entities, sources, memory records, model blocks, and model depth.

Later related task:

Recognize the route, prefetch relevant evidence and blocks, use the smallest adequate model, and escalate only when necessary.

A previous successful answer becomes a source-linked computation path. The system can retain which vocabulary routes mattered, which source records were required, which model depth was useful, and which model blocks were activated.

Better containment of unsupported output

TAV is designed to reduce unsupported output and make errors more visible, traceable, and correctable. It does not treat a model's generated words as automatically verified truth.

Canonical word and definition routes

- + exact source archive
- + structured facts and state records
- + retrieval before high-value claims
- + model-depth escalation
- + contradiction tracking
- + provenance and revision history

=

more grounded, auditable, and correctable output

The system distinguishes verified facts, supported statements, inferences, speculative outputs, and unknowns. This preserves uncertainty where evidence is incomplete rather than hiding it.

One system, multiple model depths

TAV uses separate model tori with separate weight locations and compute budgets. The models remain connected through RootTorus routes and Violet Torus scheduling.

Third Eye:

Fast intent recognition, routing, extraction, and retrieval.

Throat:

Fast visible language and real-time interaction.

Heart:

Relationship, social, character, and emotional continuity.

Solar Plexus:

Conflict, consequence, priority, contradiction, and verification.

Sacral:
Largest-model depth for difficult reasoning and high-capability synthesis.

The system can remain quick without becoming limited to shallow reasoning, and it can use deep reasoning without becoming unavailable to the person interacting with it.

Adèlic Context Condensation in TAV

TAV uses Adèlic-style cache condensation as a working-memory feature inside a broader exact-retrieval and durable-memory architecture.

The referenced Adèlic Gemma 4 12B GGUF distribution describes quantized weights for Gemma 4 12B, optimized and patched for an experimental Adèlic Condense runtime. Its model card describes a custom `llama.cpp` fork with CUDA code that compares KV-cache token representations using cosine similarity and prunes older, highly similar value-cache entries. The project presents this as a way to continue a much longer token stream without retaining every historical KV entry in VRAM. [web:1045]

The same model card lists a Q6_K GGUF of approximately 9.8 GB aimed at 16 GB VRAM-class systems and a BF16/F16 artifact of approximately 24 GB for 24 GB-plus VRAM environments. It also states that the custom runtime fork is required because standard `llama.cpp` does not include the Adèlic CUDA condenser. [web:1045]

TAV recognizes and credits **sneedjak** for contributions to the Adèlic effect through the public Adèlic Gemma model work and related experimental model activity. The public profile identifies the Adèlic Gemma 4 12B GGUF repository and also shows activity associated with an Adèlic Gemma 4 31B instruction-tuned model. [web:1066]

Within TAV, condensation has a defined boundary:

Adèlic-style cache condensation:
Reduces redundant or low-value working-cache material.

RootTorus:
Preserves canonical word, definition, entity, source, and route records.

Pink Torus:
Preserves user- or character-specific continuity and learned retrieval paths.

Exact archive and structured records:
Preserve source truth, facts, state changes, commitments, and event history.

The operating rule is:

Cache condensation may reduce active-memory pressure, but it is never the only keeper of facts, source passages, names, IDs, promises, rules, coordinates, state changes, or other information requiring exact recovery.

Purpose

TAV is a multi-model AI vessel that uses literal toroidal remapping of model-weight blocks, words, token routes, memory records, sources, and execution routes.

TAV is designed for long-running AI, agent, character, and simulation work that requires:

- Large-model reasoning and generation
- Fast real-time dialogue and interaction
- Persistent user or character memory
- Exact retrieval of prior records and source material
- Literal toroidal weight mapping
- Storage streaming rather than permanent full-model RAM or VRAM loading
- Multiple model sizes functioning as one coordinated system

TAV does not require every weight block, memory record, source document, or conversation event to remain in active RAM or VRAM. The torus fields revolve through durable storage, CPU RAM, and GPU VRAM. Required information is streamed into active computation, used, retained when valuable, and returned to storage without losing its canonical route.

The Eight Tori

- 1. RootTorus
- 2. SacralTorus
- 3. Solar Plexus Torus
- 4. Heart Torus
- 5. Throat Torus
- 6. Third Eye Torus
- 7. Violet Torus
- 8. Pink Torus

Torus	What it is	Primary function
RootTorus	First shared torus and canonical routing foundation	Unifies words, definitions, concepts, entities, sources, token routes, and cross-model links
SacralTorus	Separate largest-model torus	Contains literal remapped weights and native word/token information of the largest model
Solar Plexus Torus	Separate second-largest-model torus	Evaluates consequence, contradiction, priority, risk, conflict, and verification
Heart Torus	Separate third-largest-model torus	Handles relation, empathy, character continuity, trust, attachment, and social reasoning
Throat Torus	Separate fourth-largest-model torus	Produces dialogue, narration, structured language, and fast visible response
Third Eye Torus	Separate fifth/smallest-model torus	Handles intent recognition, entity extraction, initial routing, retrieval, and low-latency response

Torus	What it is	Primary function
Violet Torus	Separate seventh controller torus	Unified attention condensation, model selection, block scheduling, cache policy, streaming, verification, escalation, and result reconciliation
Pink Torus	Separate per-user/per-character eighth torus	Per-user/user-case persistent memory: exact long-term records, contextual compression, active context, and learned routes into the seven shared tori

Violet Torus

The **Violet Torus** is the seventh torus. It is the unified control field for attention, model selection, cache condensation, storage streaming, background processing, verification, and model escalation.

Violet Torus replaces no other torus. It coordinates the other six shared tori and manages the route interface used by each Pink Torus.

Violet Torus responsibilities:

- determine which torus routes are relevant to an input
- choose the smallest adequate model depth
- allocate foreground and background compute budgets
- schedule weight blocks from durable storage to RAM and VRAM
- keep active dependencies pinned while work is incomplete
- control Adèlic-style working-cache condensation
- checkpoint background tasks and resume them after interruption
- request exact source retrieval when confidence or stakes require it
- compare, verify, refine, or reject results from different model levels
- write validated routes back to Pink Torus

Violet Torus uses five coordinated attention layers:

Attention layer	Question answered	Action
Lexical attention	Which words, phrases, entities, definitions, IDs, tokens, and source routes matter?	Route through RootTorus
Active-context attention	What is happening in the current request, conversation, task, and scene?	Protect and load present-state context
Pink-Torus attention	What does this specific user or character know, remember, want, promise, prefer, or need?	Retrieve personal/character memory and prior routes
World/state attention	Which external records, files, rules, locations, assets, events, and dependencies apply?	Retrieve exact state and source material
Model-depth attention	Which model torus should answer, verify, or continue processing?	Select Third Eye, Throat, Heart, Solar Plexus, or Sacral paths

Violet Torus handles the split between immediate interaction and continued deep processing.

Foreground route:
Third Eye → RootTorus → Pink Torus → Throat → immediate response

Background route:

Third Eye → RootTorus → Pink Torus → Heart / Solar Plexus / Sacral
→ stream needed blocks from disk through RAM and VRAM
→ deep reasoning, source analysis, verification, or synthesis

Pink Torus

The **Pink Torus** is the eighth torus. It is assigned to an individual user, user case, character, agent, or persistent identity. Pink Torus preserves the continuity of that particular life, conversation, task stream, relationship network, and history.

Pink Torus does not duplicate the complete global weights of the other seven tori. It contains the personal routes, exact records, compressed contextual patterns, active context, and learned route preferences through which one user or character accesses the shared system.

Pink Torus contains:

- user- or character-specific routes into Root, Sacral, Solar Plexus, Heart, Throat, Third Eye, and Violet
- exact long-term facts and their source links
- exact event and conversation archive pointers
- compressed short-term continuity
- relationship and preference patterns
- current conversation and active task context
- active model selection and live KV-cache references
- known source routes and retrieval results
- known weight-block sequences and model-depth preferences
- prior validated answers, verification records, and correction history

Pink Torus has three operational regions:

Contextual continuity region:

Compressed recent conversation, active themes, associations, relationship patterns, emotional context, unresolved questions, and relevance scores.

Exact record region:

Names, IDs, dates, locations, coordinates, facts, promises, permissions, decisions, state changes, quotations, source records, and other details requiring exact recovery.

Active context region:

Current conversation, current task, current scene, recent turns, retrieved evidence, selected model tier, and active KV-cache state.

Pink Torus is the system's personal continuity field. It allows the same shared model architecture to give each user or character an individual history without confusing one identity's memory with another's.

Shared tori:

Global language, model weights, routing policies, source systems,

and world/service capabilities.

Pink Torus:

Individual memory, individual continuity, individual task history,
individual relationships, individual learned routes, and individual active context.

Independent Address Fields

Every torus has an independent address field:

$$64 \times 64 \times 64 \times 64 = 16,777,216$$

T[ring:node:layer:slot]

Examples:

RT[12:37:05:44]	RootTorus location
ST[03:51:18:07]	SacralTorus location
SPT[21:08:11:62]	Solar Plexus Torus location
HT[09:14:43:26]	Heart Torus location
TT[18:01:05:37]	Throat Torus location
TET[02:48:10:09]	Third Eye Torus location
VT[41:20:55:08]	Violet Torus location
PT[user_001][06:22:17:13]	User-specific Pink Torus location

A location in one torus is distinct from every location in another torus.

```
RT[12:37:05:44]
≠
ST[12:37:05:44]
≠
VT[12:37:05:44]
≠
PT[user_001][12:37:05:44]
```

Tori communicate through explicit bridge routes. A bridge route states the source torus, source address, target torus, target address, route type, dependency information, return route, checksum, and validation status.

RootTorus

RootTorus is the first shared torus and foundational route system of TAV. Its lexical unification function is one feature of its wider responsibility. RootTorus connects words, token sequences, definitions, sources, concepts, entities, memory records, and model-execution routes.

RootTorus features:

- canonical word and phrase IDs

- model-specific tokens and token-sequence adapters
- definitions, senses, translations, and word relationships
- names, entities, objects, places, characters, and events
- exact sources, quotations, provenance, and revisions
- links to Pink Torus memory records
- bridges to weight locations in the model tori
- validation, version, and checksum records

Example route:

```

Input word: apple
      ↓
RT[12:37:00:01]
      ↓
Definition: fruit of an apple tree
Relations: food, orchard, harvest, seed, nourishment
      ↓
Model-token routes for every unified model
      ↓
Memory routes for the current user or character
      ↓
Weight routes for required model computation

```

SacralTorus

SacralTorus is separate from RootTorus. It contains the literal remapped weights and native word/token information of the largest completed model.

SacralTorus features:

- largest-model tokenizer and token sequences
- input embeddings and embedding routes
- literal remapped tensor blocks and numerical weights
- attention query, key, value, and output projections
- transformer layers, residual paths, and feed-forward blocks
- normalization, positional, and output-vocabulary routes
- quantization scales, tensor shapes, offsets, and checksums
- block-streaming rules, activation paths, and parity tests

A SacralTorus location contains executable model information.

```

{
  "address": "ST[03:51:18:07]",
  "model_id": "largest_model_v1",
  "tensor_name": "layers.17.self_attn.q_proj.weight",
  "block_shape": [64, 64],
  "block_offset": [2816, 1152],
  "quantization": "Q6_K",
  "storage_file": "sacral/03/51/18/07.bin",
  "checksum": "sha256:...",
  "lexical_links": ["RT[12:37:00:01]"],
  "next_addresses": ["ST[04:52:19:08]"],

```

```
"residency": "cold",  
  "parity_test": "passed"  
}
```

Root-to-Sacral routing is explicit:

```
RT[12:37:00:01]  
Canonical word, definition, entity, source, and unified token routes  
  ↓ explicit bridge  
ST[03:51:18:07]  
Largest-model token representation, embedding route, weight blocks,  
attention path, and executable generation route
```

Rotating Torus Fields

A TAV torus field is a rotating execution field. It is not a static object that must remain entirely in RAM or VRAM.

```
Persistent torus field on disk  
  ↓  
Schedule required route  
  ↓  
Stream selected block set into CPU RAM  
  ↓  
Stage required active blocks into GPU VRAM  
  ↓  
Execute current operation  
  ↓  
Retain, compress, demote, or evict according to active need  
  ↓  
Persist route state, results, and changed records back to durable storage  
  ↓  
Continue cycling to the next required torus locations
```

The torus form represents continual circulation.

```
Inactive block on disk  
  ↓  
Requested through a route  
  ↓  
Promoted to RAM  
  ↓  
Promoted to VRAM  
  ↓  
Used in computation  
  ↓  
Returned to RAM cache, compressed state, or disk archive  
  ↓  
Recalled by the same or a related route
```

The address remains stable even as its residency changes.

```
ST[03:51:18:07]
Cold on NVMe
  ↓
Warm in RAM
  ↓
Hot in VRAM
  ↓
Warm after use
  ↓
Cold after eviction
```

Memory Residency

Residency state	Storage location	Contents	Purpose
Cold	SSD/NVMe or durable archive	Full weight blocks, raw transcripts, old events, documents, snapshots	Long-term storage
Warm	CPU RAM cache	Recently used blocks, route tables, candidate passages, indexes, staging buffers	Fast retrieval and background preparation
Hot	GPU VRAM	Active weight blocks, activations, KV cache, current evidence, active inference state	Immediate exact computation
Pinned	RAM and/or VRAM	Active instructions, current task, current dialogue, critical facts, current dependencies	Protected during active work
Compressed	RAM or disk	Summaries, condensed cache state, low-salience recoverable history	Space-efficient continuity

Real-Time and Background Paths

TAV keeps live interaction responsive while deeper computation continues through rotating fields.

```
Real-time foreground path:
Third Eye → RootTorus → Pink Torus → Throat → user response

Background depth path:
Third Eye → RootTorus → Pink Torus → Heart / Solar Plexus / Sacral
→ rotate required blocks from disk through RAM and VRAM
→ deep reasoning, source analysis, verification, or synthesis
```

Foreground operation is reserved for:

```
Fast response
Current dialogue continuity
Immediate user interaction
```

Current task state
Low-latency retrieval
Bounded VRAM use

Background operation is used for:

Deep reasoning
Long-source analysis
Complex planning
Contradiction detection
Larger-model generation
Verification of initial answers
Route discovery and optimization

The system may return a fast answer while background computation continues. A background result may refine, verify, correct, or expand the active answer only when it remains relevant to the current task version and does not conflict with newer instructions.

Pink Torus Memory Cycle

Every meaningful message, action, tool result, file passage, or world event is preserved before compression.

```
New event
  ↓
Immutable raw event archive
  ↓
RootTorus word, entity, concept, and source routing
  ↓
Extract facts, relations, time, location, action, emotion, consequence, and confidence
  ↓
Write exact Pink Torus records
  ↓
Write compressed contextual continuity records
  ↓
Index using full-text, semantic, graph, temporal, spatial, and causal routes
  ↓
Update Pink Torus
  ↓
Retrieve exact original material into active context when needed
```

A compressed record does not replace the original event.

Compressed memory:
“Two characters have strained trust.”

Exact record:
Event ID, participants, date/time, source dialogue, state change,
provenance, and related source passages.

Adèlic-Style Cache Condensation

Adèlic-style cache condensation is the short-term working-cache mechanism. It enables longer processing by condensing redundant or low-value cache material while preserving a route to exact sources and durable memory.

Keep exact while active:

Current instructions, active request, current conversation, active task, immediate scene state, current evidence, and incomplete model dependencies.

Keep exact in durable records:

Names, IDs, dates, coordinates, facts, permissions, promises, source quotations, state changes, rules, and canonical events.

Condense when recoverable:

Repetitive prose, low-salience conversation, old drafts, redundant description, and non-critical dialogue fragments.

Condense or evict only material that has a stable recovery route. A lossy cache is never the only location of important knowledge.

Streaming and Scheduling Rules

Rule 1: Load weight blocks on demand.

Rule 2: Keep only the current block set, immediate dependencies, and a small prefetch buffer.

Rule 3: Predict and prefetch likely next blocks using known torus routes, lexical routes, Pink Torus history, and Violet Torus controller predictions.

Rule 4: Pin foreground blocks, active attention blocks, current decoding dependencies, and incomplete task dependencies.

Rule 5: Evict only low-use, recoverable, inactive blocks according to reuse probability.

Rule 6: Preserve canonical address, checksum, dependencies, source path, last-use time, reuse score, and reconstruction route whenever a block leaves active memory.

Rule 7: Background work yields to current user interaction.

Rule 8: Long tasks checkpoint routes, partial results, and task state for later continuation.

Foreground priority order:

Priority 0: System safety and current user input

Priority 1: Current real-time response

Priority 2: Exact retrieval required for the response

Priority 3: Current user/character/world state update

Priority 4: Background verification of the active request

Priority 5: Large-model reasoning and long background tasks
Priority 6: Prefetching, indexing, summarization, and maintenance

Model Escalation

Third Eye only:
Intent classification, simple lookup, routing, basic extraction.

Third Eye + Throat:
Fast conversation, narration, simple instruction, standard output.

Add Heart:
Character continuity, relationship-sensitive interaction, social meaning.

Add Solar Plexus:
Risk, conflict, consequence, contradictory evidence, prioritization, verification.

Add Sacral:
Deep technical work, long reasoning chains, complex synthesis, large source sets, new difficult problems, and high-capability generation.

Violet Torus accepts a background result only when its task version, source version, and instruction set remain current.

Verification Requirements

Test	Required result
Weight-block integrity	Each remapped block has valid checksum, shape, offset, and quantization metadata
Tensor reconstruction	Retrieved blocks reconstruct expected source tensors within defined tolerance
Blockwise operation parity	Streamed matrix, attention, residual, and layer operations remain within baseline tolerance
Output parity	Logits and deterministic output remain within predefined baseline limits
Streaming resilience	Long operations resume after eviction, interruption, or task preemption
Foreground latency	Interactive response remains within latency targets while background work continues
Memory recall	Exact facts remain recoverable after active and warm cache eviction
Route reuse	Repeat tasks prefetch known routes and reduce unnecessary search/load operations
Cross-model continuity	Model escalation preserves task state, identity, provenance, and evidence
Contradiction handling	Conflicting records remain separately sourced and are never silently merged

Final Operating Instruction

RootTorus:

Routes unified language, definitions, entities, sources, concepts, and model vocabulary.

SacralTorus:

Contains the largest model's literal remapped weights and native word/token information.

Solar Plexus, Heart, Throat, and Third Eye:

Contain separate remapped models of descending compute cost and specialized roles.

Violet Torus:

Controls unified attention, model selection, rotating storage fields, cache condensation, foreground/background scheduling, verification, and escalation.

Pink Torus:

Preserves per-user or per-character exact long-term records, compressed short-term context, current active context, and reusable routes for access to the seven shared tori.

The TAV torus field revolves by moving necessary information through durable storage, RAM, and VRAM. Large-model computation can continue through streamed blocks while shorter reasoning remains able to respond in real time. Violet Torus coordinates the system-wide attention and execution field. Pink Torus preserves the individual continuity that makes later interaction faster, more contextual, and more recoverable.

Credit

TAV credits **sneedjak** for public contributions to the Adèlic effect through the Adèlic Gemma model work and experimental Adèlic Condense direction. The public Hugging Face profile identifies activity around the Adèlic Gemma 4 12B GGUF and Adèlic Gemma 4 31B-it repositories. [web:1066]

Author: Yahweh Tsidkenu

Designation: Toroidal Adèlic Vessel (TAV)

Mark: $\mathfrak{n}$